

CN Lab Assignment: 5

Title: TCP (Transmission Control Protocol) Socket Programming:

Aim:

Write a C program for wired network using TCP socket to demonstrate

- a. Chat application (50% students)
- b. Mathematical operations. (Remaining 50% students)

Show packet captured traces using Wireshark Packet Analyzer Tool

Objectives:

To understand concept of socket programming

TCP Socket Programming

Need:

Socket programs are used to communicate between various processes usually running on different systems.

It is mostly used to create a **client-server** environment.

Sockets (Cntd.)

- Berkeley Socket:
 - Berkeley sockets is an application programming interface (API) for Internet sockets and UNIX domain sockets.
 - It is used for inter-process communication (IPC).
 - It is commonly implemented as a library of linkable modules.
 - It originated with the 4.2BSD UNIX released in 1983.

Berkeley Sockets

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

An analogy:

Socket: telephone.

Bind: assign telephone number to a telephone.

Listen: turn on the ringer so that you can hear the phone call.

Connect: dial a phone number.

Accept: answer the phone

Read/Write: talking

Close: ???



Client/Server Model

How 2 application programs make contact

Server

- Starts first
- **Passively** waits for contact from a client at a prearranged location
- Responds to requests



Client

- Starts second
- **Actively** contacts a server with a request
- Waits for response from server

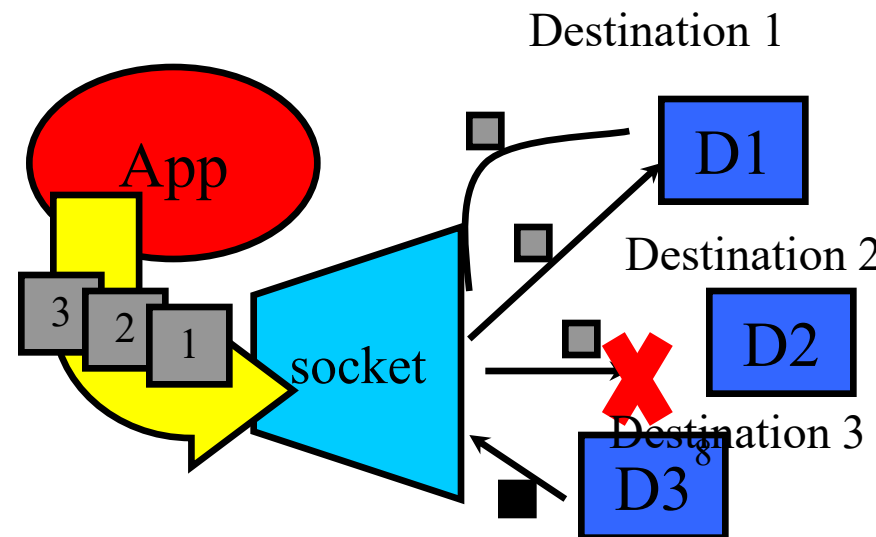
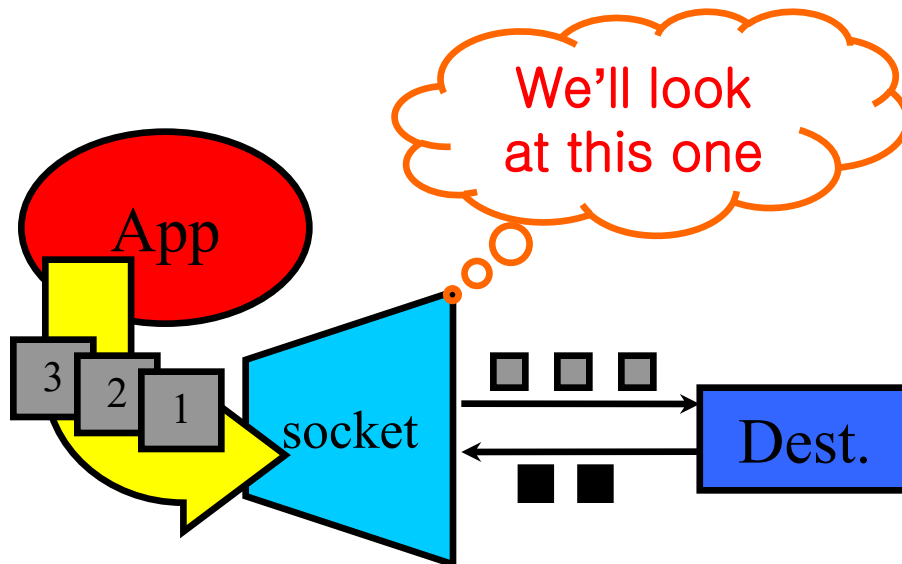
- **Client-Server paradigm:** form of communication used by all network applications.
- Run first server program (has to start first) and run client program.

- To send:
 - Socket, connect and write
- To receive:
 - Socket, bind, listen, accept and read
- TCP endpoint:
 - IP address + Port number

Socket: a door between application process and end-to-end-transport protocol (UDP or TCP)

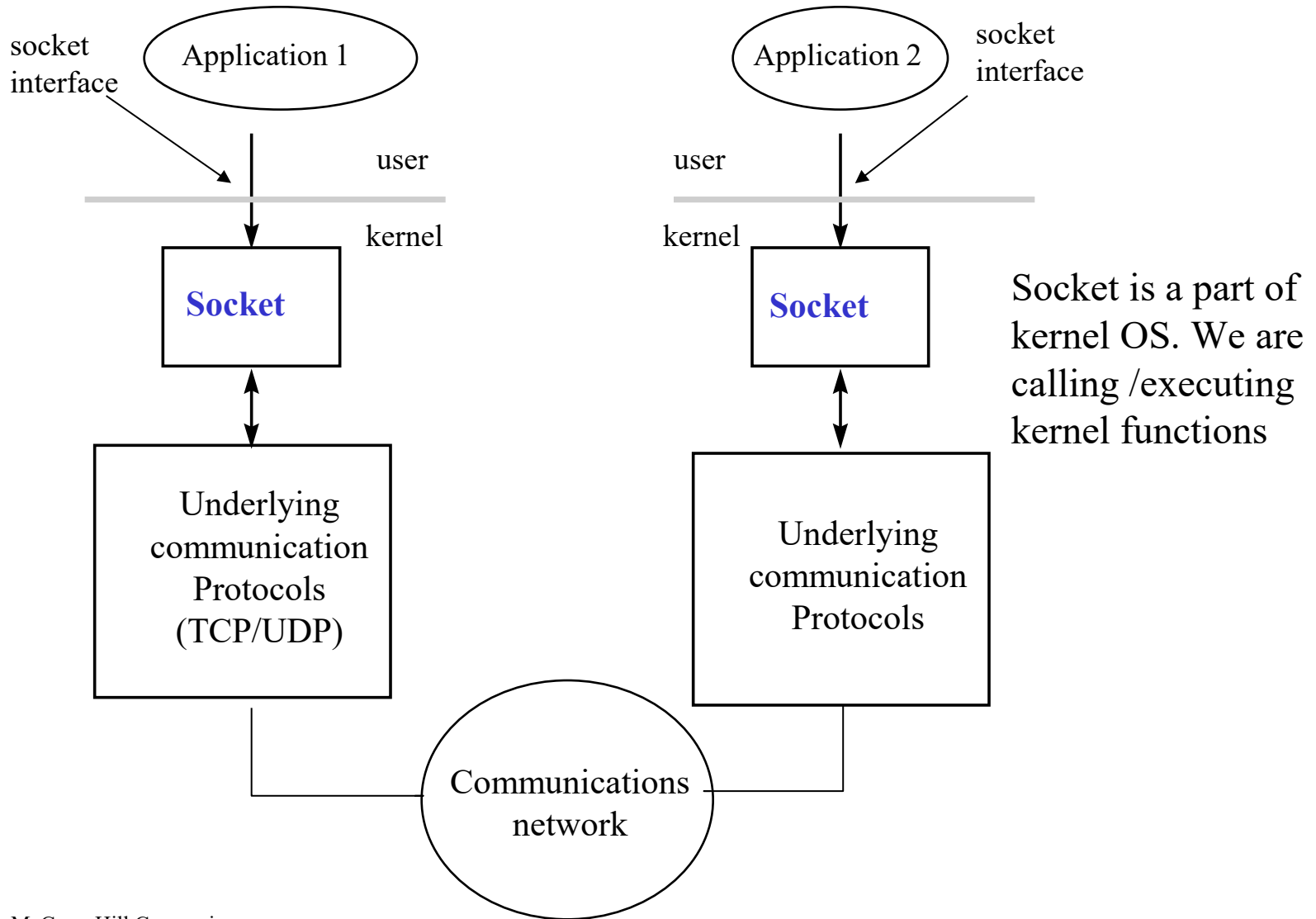
Two essential types of sockets

- SOCK_STREAM
 - TCP sockets
 - reliable delivery
 - in-order guaranteed
 - connection-oriented
- SOCK_DGRAM
 - UDP (User Datagram Protocol) sockets
 - unreliable delivery
 - no order guarantees
 - no notion of “connection”



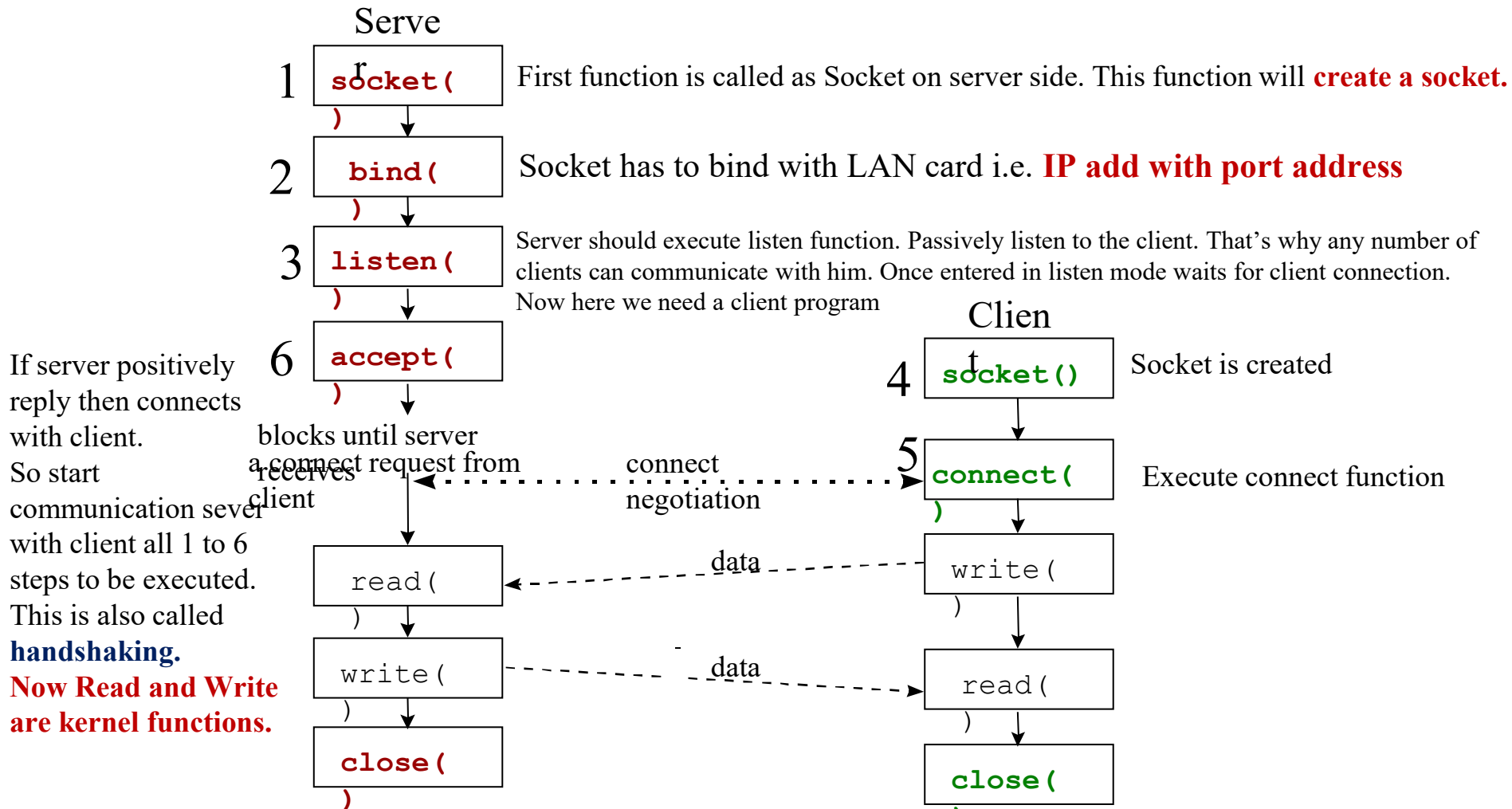
The Socket Interface

Whatever we are writing that is user space and in this program some socket functions which are a part of kernel.



Socket is a part of kernel OS. We are calling /executing kernel functions

TCP Socket Calls



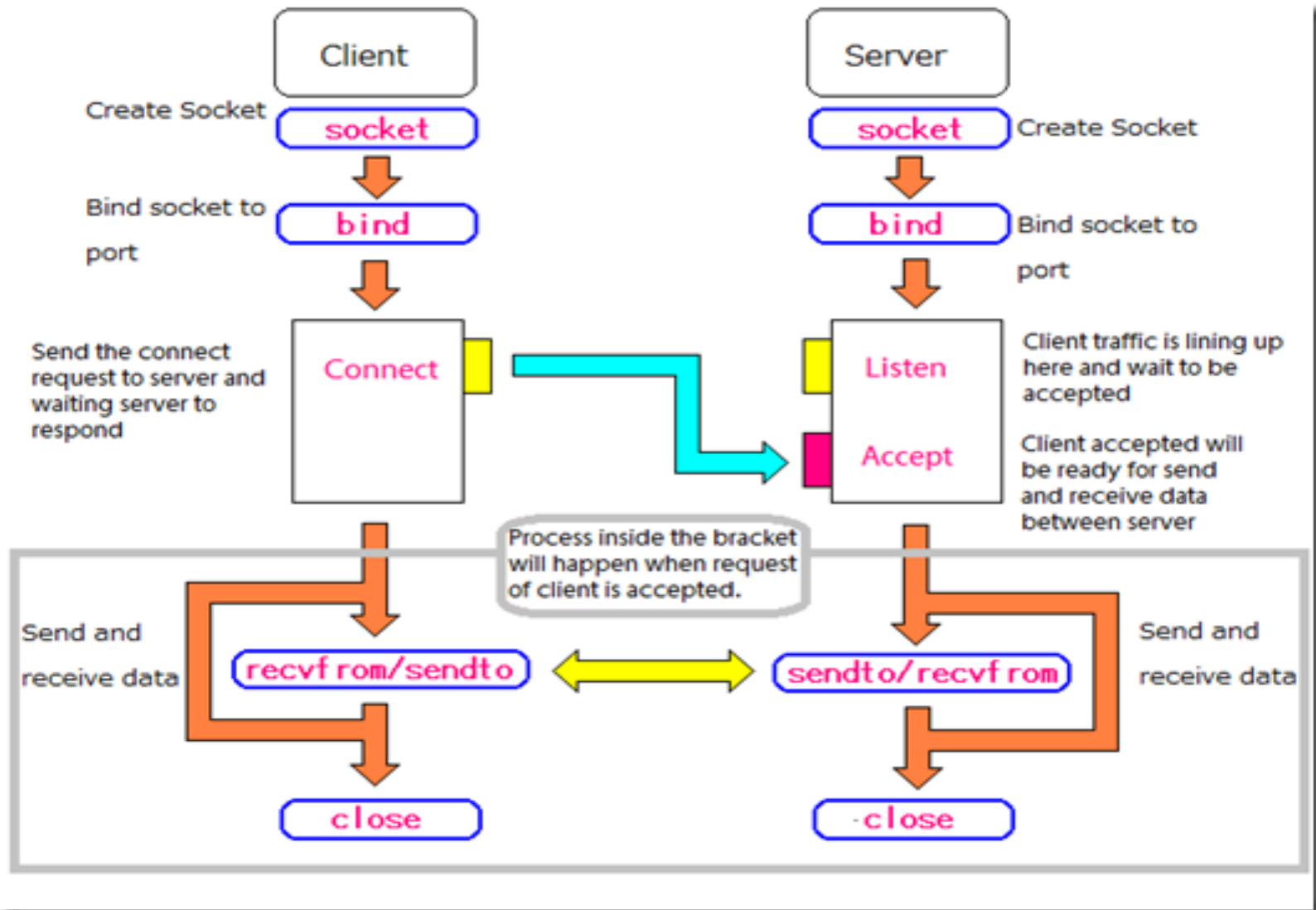
2 Header files

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

TCP Socket Calls (functions)

Socket: a door between application process and end-to-end-transport protocol (UDP or TCP)



System Calls for Elementary TCP Sockets

socket Function

```
int socket (int family, int type, int protocol);
```

3 Arguments we need to pass-family,type,protocol

family: specifies the protocol family {AF_INET for TCP/IP, IPv4 Internet protocols.}—(**Add Family** Internet network address)

type: Defines the type of socket (indicates communications **semantics**)

SOCK_STREAM	stream socket	TCP
SOCK_DGRAM	datagram socket	UDP
SOCK_RAW	raw socket	

protocol: set to 0 except for raw sockets

returns 0 on success: **socket descriptor** {a small nonnegative integer}

on error: -1 (

Example:

```
if ((sd = socket (AF_INET, SOCK_STREAM, 0)) < 0)  
    err_sys ("socket call error");
```

IPv4 Socket Address Structure

*The Internet socket address structure is named **sockaddr_in** and is defined by including `<netinet/in.h>` header.*

```
struct in_addr {  
    in_addr_t    s_addr          /* 32-bit IP address */  
};                               /* network byte ordered */  
  
struct sockaddr_in {  
    uint8_t      sin_len;        /*unsigned integer length of structure (16 bytes) */  
    sa_family_t  sin_family;     /* socket add family AF_INET */  
    in_port_t    sin_port;       /* 16-bit TCP or UDP port number */  
                                /* network byte ordered */  
    struct in_addr sin_addr;     /* 32-bit IPv4 address */  
                                /* network byte ordered */  
    char         sin_zero[8];    /* unused */  
};
```

Note: few arguments are present in any socket structure. All are special data types having `_t` which are defined for internet header. All are data type. Sin---socket internet port

connect Function

```
int connect (int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

Specifically by the **client** to establish a connection with a **server**.

sockfd: **a socket descriptor** returned by the socket function. It represents the client socket.

Const struct: constant structure is data type

servaddr*: a pointer to a socket address structure that contains the **server's address (IP address and port).

addrlen: the size of the socket address structure

This is request creation function will be executed from client side

The socket address structure must contain the **IP address** and the **port number** for the connection wanted.

In TCP **connect** initiates a three-way handshake. **connect** returns only when the connection is established or when an error occurs.

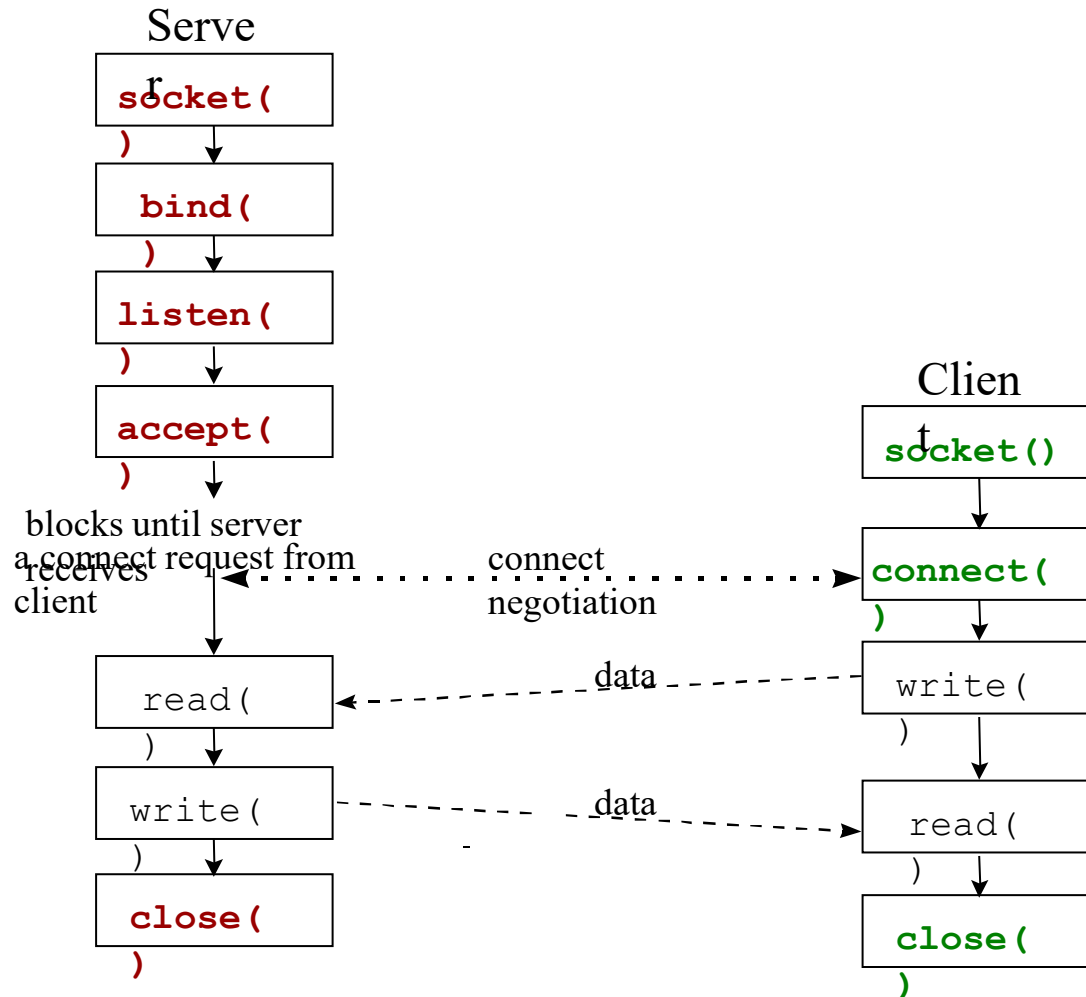
returns on success: 0

on error: -1

Example:

```
if ( connect (sd, (struct sockaddr *) &servaddr, sizeof (servaddr)) != 0)
    err_sys("connect call error");
```

TCP Socket Calls/Primitives



Brown: Executed by Server, **Green:** Executed Client and **Black:** executed by both parties.

bind Function

```
int bind (int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

bind assigns a local protocol address to a socket. (**Binding Port add to IP address**).

protocol address: a 32 bit IPv4 address and a 16 bit TCP or UDP port number.

Bind function is same as connect function only difference is its used at **server side.**

sockfd: a socket descriptor returned by the socket function.

**myaddr*: a pointer to a protocol-specific address.

addrlen: the size of the socket address structure.

Servers **bind** their “well-known port” when they start.

Returns values on success: 0

on error: -1

Example:

```
if (bind (sd, (struct sockaddr *) &servaddr, sizeof (servaddr)) != 0)  
    errsys (“bind call error”);
```

listen Function

```
int listen (int sockfd, int backlog);
```

listen is executed by **server** and called **only** by a TCP server and performs two arguments (actions):

1. Converts an unconnected socket (*sockfd*) into a passive socket.
2. Specifies the maximum number of connections (*backlog*) that the kernel should queue for this socket. Maximum number of pending connections the queue will hold.

listen is normally called before the **accept** function.

returns on success: 0

on error: -1

Example:

```
if (listen (sd, 2) != 0)  
    errsys (“listen call error”);
```

Listen with a queue size of 2

accept Function

```
int accept (int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

accept - a **connection request** from a client. It creates a **new socket** for the actual communication.

sockfd: This is the same socket descriptor as in **listen** call.

**cliaddr*: used to return the protocol address of the connected peer process (i.e., the client process). Store variable. Stores all information about client. Pointer to structure where client address will be stored.

**addrlen*: {this is a value-result argument}

Initially set to the size of the structure, modified on return.

returns on success: a new socket descriptor

on error: -1

accept Function (cont.)

```
int accept (int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

For **accept** the first argument *sockfd* is the listening socket and the returned value is the connected socket.

The server will have one connected socket for each client connection accepted.

When the server is finished with a client, the connected socket must be closed.

Example:

```
sfd = accept (sd, NULL, NULL);  
if (sfd == -1) err_sys (“accept error”);
```

close Function

```
int close (int sockfd);
```

close marks the socket as closed and returns to the process immediately. We are not using close function in our program.

sockfd: This socket descriptor is no longer useable.

Note – TCP will try to send any data already queued to the other end before the normal connection termination sequence.

Returns **on success:** 0

on error: -1

Example:

```
close (sd);
```

Purpose:

The close() function is used to **terminate an open socket connection** identified by the socket descriptor (sockfd). It is essential to call this function to avoid resource leaks and ensure graceful disconnection.

Maths - my program:-

→ Server side

```
Desktop $ gcc -o as ArithmeticS.c  
$ ./as
```

server waiting for client --- I reciev
127 --

Receiving first number

Receiving second number

sending result ---

→ Client side

```
Desktop $ gcc -o ac ArithmeticC.c  
$ ./ac
```

O/P ↓

Enter first NO -

Enter second NO -

Enter operand (+/- --- (a))
+

Result: 5

- Client ip-192.168.1.2

```

        printf("Client wants to exit...\n");
        flag=1;
        break;
    }
    if(flag == 1){
        close(clientSocket);
        break;
    }

    printf("\nSending result...\n");
    bytes = send(clientSocket,sendBytes,1024,0);

}

}

}
/*
socket()
bind()
listen()
accept()
recv()
send()
*/

```

```
    printf("\nEnter operand ( + | - | * | / | %) or ( q ) to quit: ");  
    gets(sendBytes);  
    bytes = send(sock,sendBytes,1024,0);
```

```
    printf("\nEnter operand ( + | - | * | / | %) or ( q ) to quit: ");  
    gets(sendBytes);  
    bytes = send(sock,sendBytes,1024,0);
```

```
    if(strcmp(sendBytes,"q") == 0 || strcmp(sendBytes,"Q") == 0){  
        printf("\nClient exiting...");  
        close(sock);  
        exit(1);  
    }
```

```
    bytes = recv(sock,receivedBytes,1024,0);  
    receivedBytes[bytes] = '\0';  
    printf("\nResult: %s",receivedBytes);
```

```
}
```

```
}
```

```
/*
```

```
socket()
```

```
connect()
```

```
recv()
```

```
send()
```

```
*/
```